CS543 · WEB SERVICES · ASSIGNMENT 6

Smart Contact Manager (SCM)

Honest responses: status codes, errors and validation

Team ID	12
Team members	Aman Bali — 20251651015 Bhupesh Kumar — 20251651032
Technology	Java 21 · Spring Boot 3.2.5 · Maven · Spring Web · Jakarta Bean Validation · JUnit 5 · MockMvc
Service port	8090 (same Assignment 5 service; existing SCM Contact Service: 8082)

This submission updates the same Spring Boot service from Assignment 5. The original handout describes an orders domain; this report adapts the required HTTP/error/validation behaviors to SCM contact-share resources rather than introducing an unrelated orders service.

CS543 Assignment 6 — Honest Responses: Status Codes, Errors & Validation

Course: CS543 — Web Services

Team ID: 12

Aman Bali — 20251651015

Bhupesh Kumar — 20251651032

1. SCM adaptation and implementation scope

The supplied assignment handout describes an `/orders` service and commerce-specific failures (`empty-cart`, `item-unavailable`, `payment-declined`, `order-not-found`, `illegal-transition`). Smart Contact Manager does not own an orders/payment domain. To keep this submission **with SCM only**, Assignment 6 applies the same required HTTP/error/validation behaviors to the existing Assignment 5 `contact-share-service` and its `/v1/contact-shares` resources.

This is the **same Spring Boot service and port 8090 from Assignment 5**, not a new microservice. It calls the existing SCM Contact Service on port 8082. It is implemented in **Java 21, Spring Boot 3.2.5, Spring Web, Jakarta Bean Validation, Maven, JUnit 5, Mockito and MockMvc**. No Python/Flask code is used.

2. Status map (A1)

Endpoint	Success	Failure statuses and reasons
`POST /v1/contact-shares`	`201 Created` with `Location` and representation	`400` malformed JSON/missing Idempotency-Key; `405` unsupported method; `415` unsupported request Content-Type; `401` missing/invalid demo bearer token; `406` unsupported Accept; `409` same key reused with a different body; `422` invalid fields or referenced contact missing; `429` client budget exceeded; `502` invalid downstream response; `503` Contact Service unavailable
`GET /v1/contact-shares`	`200 OK` list	`401` auth; `406` unsupported Accept; `422` invalid filter/sort/order; `429` rate limit
`GET /v1/contact-shares/{shareId}`	`200 OK` representation, or `304 Not Modified` when ETag matches	`401` auth; `404` unknown share; `406` unsupported Accept; `429` rate limit
`PATCH /v1/contact-shares/{shareId}`	`200 OK` updated representation and new ETag	`400` malformed/empty patch; `405` unsupported method; `415` unsupported request Content-Type; `401` auth; `404` unknown share; `406` unsupported Accept; `409` share not pending; `412` stale If-Match; `422` field validation; `428` missing If-Match; `429` rate limit
`DELETE /v1/contact-shares/{shareId}`	`204 No Content`	`401` auth; `404` unknown share; `406` unsupported Accept; `429` rate limit
`POST /v1/contact-shares/{shareId}/cancellation`	`200 OK` with cancelled resource	`401` auth; `404` unknown share; `406` unsupported Accept; `409` transition not allowed; `429` rate limit
`OPTIONS` collection/item/cancellation	`204 No Content` and `Allow`	OPTIONS is exempt from demo authentication to support CORS preflight

`200 OK` is never used to represent a failed operation. A successful create returns `201 Created`; deletion returns `204 No Content`; a conditional cache hit returns `304 Not Modified`.

3. Retry-After and retry safety (A4)

`429 Too Many Requests` and downstream `503 Service Unavailable` include `Retry-After: 30`. The service does not tell clients to retry permanent client errors.

• **Safe to retry:** GET collection, GET item, OPTIONS. These methods do not mutate a share.

• **Create:** retry only with the **same** `Idempotency-Key` **and identical normalized request body**. The original resource is replayed. Reusing the key with a different body returns `409 Conflict`.

• **PATCH:** not blindly retryable after an uncertain outcome; GET the resource again and use the current ETag in `If-Match`.

• **DELETE:** semantically idempotent, but a repeated request returns `404` after the first successful delete.

• **Cancellation:** after an uncertain response, read the resource state; another cancellation returns `409` because only `pending` can transition to `cancelled`.

• Do not retry unchanged `4xx` requests. Correct the request/authentication/state first.

4. Problem Details and type catalogue (B1–B2)

Every failure uses the same four required fields (`type`, `title`, `status`, `detail`) and an `errors` array. The response media type is `application/problem+json;charset=UTF-8`. `errors` is empty for non-field errors. Internal exception messages and stack traces are logged server-side and are not returned to clients.

Stable SCM-specific type catalogue:

Type string	Trigger						
`invalid-request`	Malformed JSON or missing/invalid required request header						
`invalid-share-request`	Request fields fail validation; includes `errors[]`						
`contact-unavailable`	Contact reference is validly formatted but does not exist in SCM Contact Service						
`share-not-found`	Requested share ID does not exist						
`illegal-transition`	Update/cancel attempted when the share is not pending						
`idempotency-conflict`	Same idempotency key is used for a different payload						
`invalid-share-filter`	Unsupported status/sort/order query						
`authentication-required`	Missing or invalid bearer token						
`not-acceptable`	Client requests a response format the endpoint does not provide						
`rate-limit-exceeded`	Per-client rate limit is exceeded						
`contact-service-error`	Downstream Contact Service returns a mapped bad-gateway failure						
`dependency-unavailable`	Downstream Contact Service is unavailable; retry may be appropriate						
`stale-resource-version`	`If-Match` does not match current ETag						
`precondition-required`	PATCH has no `If-Match` header						
`method-not-allowed`	HTTP method is not supported for this resource	\n	`unsupported-media-type`	Request Content-Type is not supported	\n	`internal-error`	Unexpected server failure; client receives no internal detail

The handout's commerce terms are intentionally not copied as if SCM processes orders or payments. The closest SCM concepts are mapped to the same stable error-catalogue idea: empty/invalid share input → `invalid-share-request`, unavailable referenced

contact → `contact-unavailable`, missing resource → `share-not-found`, invalid state transition → `illegal-transition`.

5. Example Problem Details body (B3)

A request with `contactRef` blank and `recipientEmail` invalid returns one `422` response with all detected field errors:

```
{
  "type": "invalid-share-request",
  "title": "Unprocessable Content",
  "status": 422,
  "detail": "One or more fields failed validation.",
  "errors": [
    {"field": "contactRef", "reason": "must not be blank"},
    {"field": "recipientEmail", "reason": "must be a well-formed email address"}
  ]
}
```

• `type`: stable machine-readable category.

• `title`: short human-readable summary.

• `status`: HTTP status code repeated in the body.

• `detail`: safe summary; it does not expose database/framework internals.

• `errors[]`: each field and validation reason. The exact order/messages depend on Bean Validation provider output.

6. Validate before acting (C1–C2)

`CreateShareRequest` is validated using Jakarta Bean Validation before controller business logic runs:

• `contactRef` is required, nonblank and at most 255 characters.

• `recipientEmail` is required, nonblank, valid email format and at most 255 characters.

• `message` is optional and at most 500 characters.

• `Idempotency-Key` is required and must be 1–128 characters.

• For PATCH, at least one update field must be present; email must be valid if provided and message is limited to 500 characters.

All Bean Validation field errors are collected into one `422` Problem Details response. Malformed JSON is a syntax/parsing error and returns `400`. The OpenAPI contract documents the same constraints. SCM contact-share requests do not contain an `items[]`, quantity, delivery address, cart or payment field; those are specific to the handout's orders domain and would be misleading to add to SCM merely to copy the example schema.

7. Content negotiation and response formats (C3–C4)

• Default success representation: `application/json;charset=UTF-8`.

• This implementation does **not** serialize successful resources as XML. `Accept: application/xml` therefore returns `406 Not Acceptable` with a Problem Details body.

• Failures use `application/problem+json;charset=UTF-8`, including malformed input, auth failures, missing resources, rate limiting and dependency errors.

• `204` and `304` responses intentionally have no body.

• The response content type is explicitly labelled by the response advice/error handler.

8. Internal error mapping and hidden details (B4)

`GlobalExceptionHandler` maps expected API errors and dependency errors to safe Problem Details. Unexpected exceptions are logged server-side with the exception for diagnosis; the caller receives only `500 Internal Server Error` and `The server could not complete the request.` The client never receives a stack trace, database message, filesystem path, secret, or framework error string. The Contact Service client uses bounded connect/read timeouts and bounded retries with backoff for transient transport/server errors, not for ordinary 4xx responses.

9. Curl evidence (D1–D4)

Run the script after starting both SCM Contact Service (8082) and this Assignment 6 service (8090). Supply an actual contact ID:

```
CONTACT_REF='your-real-scm-contact-id' ./capture-curl-transcript.sh
```

It captures:

• success create: `201`, `Location`, JSON body;

• repeat create with the same key: idempotent replay;

• invalid fields: one `422` with multiple `errors[]`;

• malformed JSON: `400`;

• unknown share: `404`;

• illegal transition: cancel a share twice and observe `409`;

• XML Accept: `406`;

• missing authentication: `401`;

• `503`/`Retry-After` when Contact Service is unavailable;

• a rate-limit demonstration when started with a small `RATE_LIMIT_PER_MINUTE` value;

• conditional GET and stale `If-Match` to re-check Assignment 5 behavior.

The committed `transcript.txt` is a checklist, not fabricated evidence. The generated `transcript-actual.txt` must be captured against your running services before submission.

10. Assignment 5 regression checks (D4)

The test suite includes checks for create/Location, idempotency replay, conditional GET/304, stale If-Match/412, malformed JSON, missing resources, invalid multi-field input/422, XML Accept/406, missing authorization/401 and dependency failure/503 with `Retry-After`. Run the tests in the environment used for submission and retain actual output.

11. Run

Java 21 and Maven are required. Start the existing SCM Contact Service first:

```
export CONTACT_SERVICE_URL="http://localhost:8082/v1/contacts"
mvn clean test
mvn spring-boot:run
```

The service stays on port `8090`; this is an update of Assignment 5's same service, not a new service. The demo bearer token is configured by `SCM_DEMO_BEARER_TOKEN` and is only a header-handling demonstration, not production authentication.